

BIG DATA · PROCESSAMENTO DE DADOS MASSIVOS (PDM)

Perfilamento Paralelo de Grandes Datasets CSV

Escalando o módulo `DataSetSummary` do framework **VisFlow-MM**

Python `ProcessPoolExecutor` vs. Apache Spark `local[N]` · NYC Yellow Taxi (1 · 5 · 10 GB)

PPGCC — UFLA

2026/1

AMD Ryzen 7 8700G · 16 núcleos lógicos

1 · Contexto científico

NL2Vis, o framework VisFlow-MM e o papel do pré-processamento determinístico

2 · O módulo `DataSetSummary`

As 7 etapas determinísticas e o gargalo de escala (`MemoryError`)

3 · Problema & perguntas de pesquisa

RQ1 · RQ2 · RQ3 — escalar sem perder exatidão nem romper a interface

4 · Solução — pipeline paralelo

Multiprocessing Python *vs.* Apache Spark · chunks · agregação exata

5 · Experimentos

NYC Yellow Taxi (1 · 5 · 10 GB) · ambiente · protocolo de benchmark

6 · Resultados

Multiprocessing não escala · Spark atinge 5,35× com 16 núcleos

7 · Análise — Lei de Amdahl

Por que um escala e o outro não · recomendações para produção

8 · Conclusões & trabalhos futuros

Apresentação · ~30 min + perguntas · PPGCC/UFLA 2026

De linguagem natural à visualização (NL2Vis)

LLMs impulsionaram sistemas que geram gráficos a partir de consultas em linguagem natural, eliminando a necessidade de conhecer gramáticas de visualização — Chat2Vis [Maddigan & Susnjak 2023], LIDA [Dibia 2023], NVAgent [Ouyang et al. 2025] e DeepVIS [Shuai et al. 2026].

VisFlow-MM

[Fonseca 2026b] — sistema NL2Vis híbrido: pré-processamento determinístico → geração de código Python via LLM (KIMI-K2.5) → validação por agentes → condicionamento multimodal por imagem de referência.

Tese central:

em sistemas NL2Vis o LLM *nunca* acessa os dados brutos — recebe uma **representação intermediária** construída por um módulo determinístico. A qualidade e a escalabilidade desse pré-processamento determinam o *teto de desempenho de todo o pipeline*.

Por que perfilamento determinístico?

- CSVs reais **excedem a janela de contexto** dos modelos
- Carregam ruído (tipos heterogêneos, datas variadas) que degrada a geração de código [Wu et al. 2024]
- LLMs têm **difículdade em calcular estatísticas precisas** sobre dados brutos [Zhu et al. 2024]

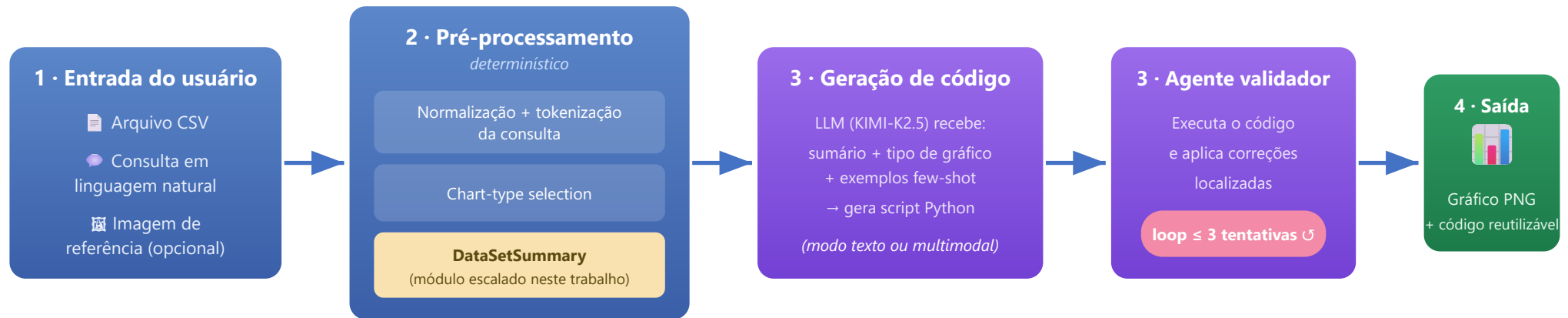
Consequência:

o módulo de perfilamento (`DataSetSummary`) não é mera infraestrutura — é um *objeto de pesquisa de primeira ordem*.

4

estágios do pipeline VisFlow-MM
(detalhados no próximo slide)

Pipeline do VisFlow-MM em 4 Estágios



Foco deste trabalho:

o gargalo está no estágio 2 — a sumarização via `DataSetSummary`. Escalá-la para datasets de escala Big Data, *sem perder exatidão estatística* e *sem romper a interface* esperada pelo pipeline, é o desafio endereçado.

Resultados no VisEval [Chen et al. 2025]

Benchmark que mede taxa de código *inválido*, saída *ilegal* e taxa de *aprovação* (pass rate). Valores em **verde** = melhor por métrica.

Método	Inválido %	Ilegal %	Pass %
VisFlow-MM (text-only)	20,40	18,50	61,20
VisFlow-MM (multimodal)	0,13	19,69	80,18
LIDA [Dibia 2023]	1,13	21,20	77,66
Chat2Vis [Maddigan & Susnjak 2023]	0,86	21,37	77,75
NVAgent [Ouyang et al. 2025]	0,72	13,63	85,63

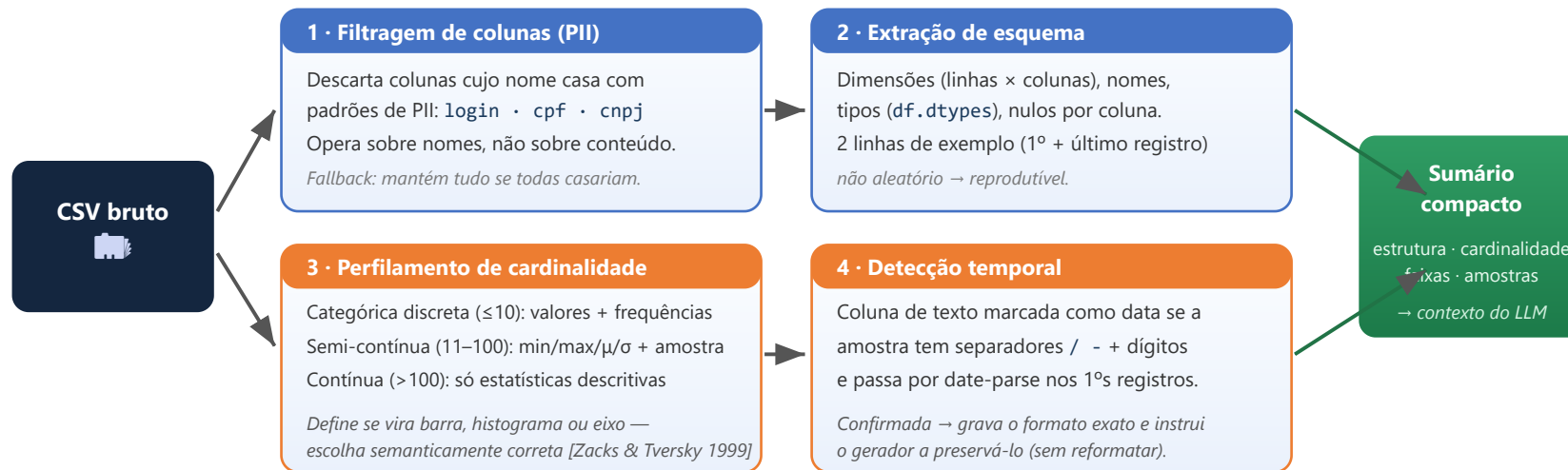
Leitura dos resultados

O **condicionamento multimodal** derrubou a taxa de código inválido de **20,40% → 0,13%** e elevou a aprovação de **61,20% → 80,18%**.

VisFlow-MM **supera LIDA e Chat2Vis** em pass rate e fica competitivo com o NVAgent — que usa arquitetura multi-agente mais complexa.

Conexão com este trabalho: esse desempenho depende de um sumário de qualidade. Se o perfilamento falha (`MemoryError` em datasets grandes), *todo o pipeline cai* — daí a urgência de escalá-lo.

DataSetSummary — As Etapas Determinísticas



Determinístico: a mesma entrada gera sempre o mesmo sumário — garantia de reprodutibilidade exigida pelo pipeline. (7 sub-etapas no total, agrupadas em 4 estágios)

O Gargalo de Escala e as RQs

O problema de escala

MemoryError:

O `DataSetSummary` original executa `pd.read_csv(path)` *sem* `chunksize` — carrega o arquivo inteiro na RAM. Um CSV de 10 GB exige ≥ 10 GB só para o DataFrame, mais as estruturas de profiling.

Datasets de dezenas de gigabytes são comuns em **transporte urbano, IoT, logs de sistema, saúde e financeiro**. A premissa de "tudo cabe na RAM" torna-se inviável em máquinas com 16–32 GB e outros processos ativos.

Restrições do desafio:

escalar *sem perder exatidão estatística* e *sem romper a interface* esperada pelo VisFlow-MM [Fonseca 2026b].

Perguntas de pesquisa

RQ1.

Adicionar *workers* Python (`ProcessPoolExecutor`) reduz o tempo de perfilamento de CSVs grandes?

RQ2.

O Apache Spark [Zaharia et al. 2016] apresenta melhor escalabilidade — e *por quê*?

RQ3.

Qual abordagem é mais adequada para integração com o VisFlow-MM em produção?

Resultado-síntese (spoiler):

comportamento contra-intuitivo — o multiprocessing Python *não escala* (gargalo de I/O), enquanto o Spark atinge **5,35× com 16 núcleos**.

~94 M

linhas no maior dataset
(10 GB em CSV)

MemError

abordagem original
(`pd.read_csv` sem chunks)

2

paradigmas de paralelismo
implementados e comparados

Por que este dataset?

- **Volume:** >10 GB em CSV, ~94 M linhas (Jan/2022–Jun/2024)
- **Variedade:** numéricas (tarifas, distância), categóricas (pagamento), temporais (embarque/desembarque) — exercita as 7 etapas
- **Público e reprodutível:** sem conta nem licença [NYC TLC 2024]
- **Controlável:** concatenando meses construímos 1, 5 e 10 GB exatos

Formato original:

Parquet (Snappy, ~47 MB/mês). **Expansão ~8×** ao converter para CSV → ~380 MB CSV/mês.

Arquivo	Meses	Linhas	Tamanho
taxi_1gb.csv	~3	~8 M	1.069 MB
taxi_5gb.csv	~14	~49 M	4.585 MB
taxi_10gb.csv	~27	~94 M	8.910 MB

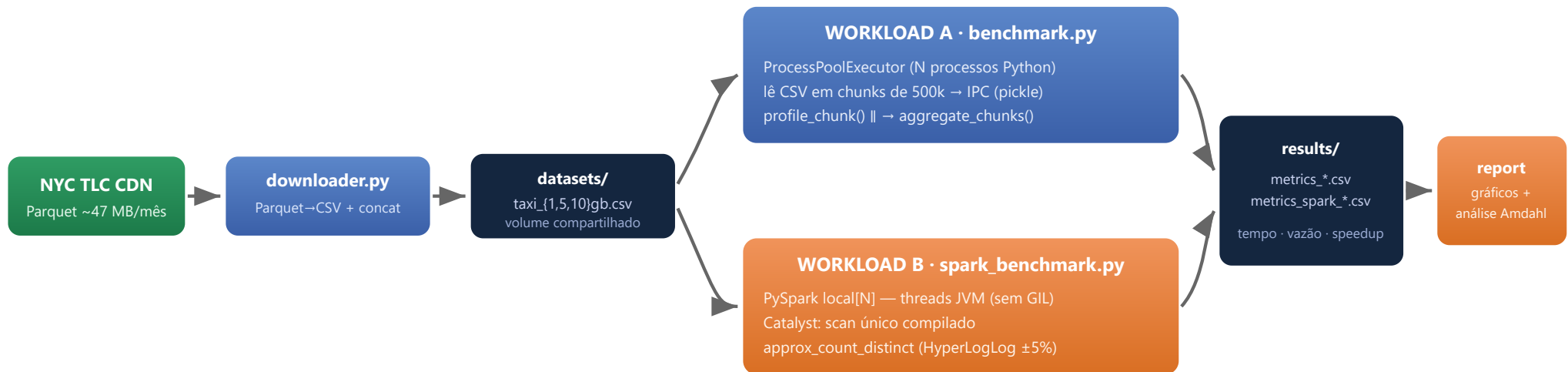
Colunas representativas (19 no total)

Coluna	Tipo	Tratamento no profiler
trip_distance	float	Contínua (>100 únicos) → histograma
fare_amount	float	Contínua → estatísticas descritivas
passenger_count	float	Semi-contínua (1–8) → bins
payment_type	int	Discreta (≤10) → barras
store_and_fwd_flag	string	value_counts (Y/N)
tpep_pickup_datetime	string	Deteccção temporal

Exemplo de decisão semântica:

sem a classificação de cardinalidade, o LLM trataria `payment_type` (6 valores inteiros) como contínua e geraria um histograma, onde um *gráfico de barras* é o correto [Zacks & Tversky 1999].

Arquitetura do Pipeline Paralelo



downloader

setup

Baixa Parquet, converte e concatena os CSVs-alvo

benchmark / spark-benchmark

experimento

Mede tempo e vazão · $N \in \{1,2,4,8,16\}$ · 3 repetições

report-all

relatório

Tudo orquestrado por `docker compose` →
reprodutível [Fonseca 2026a]

A ideia central

Substituir a leitura integral por **leitura incremental**: cada chunk tem 500.000 linhas (≈ 90 MB de RAM), *independente* do tamanho total do arquivo.

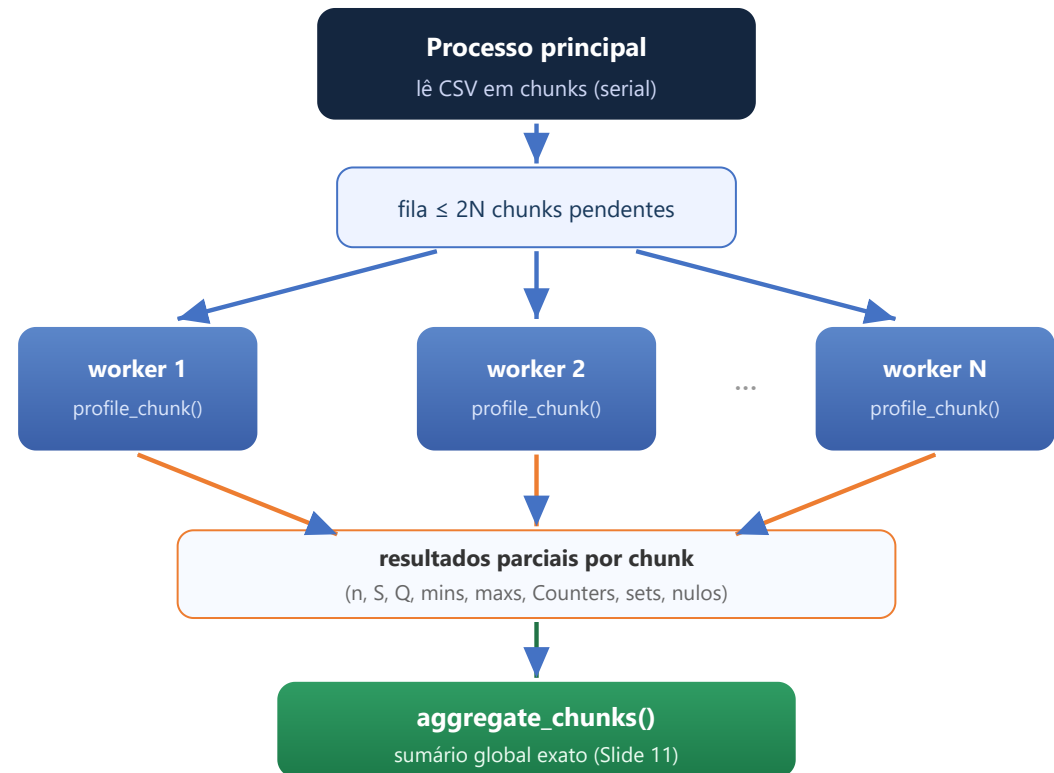
```
reader = pd.read_csv(path,  
    chunksize=500_000)  
for chunk in reader:  
    profile_chunk(chunk)
```

Janela deslizante (Listing 2):

mantém no máximo $2 \times N$ chunks pendentes. Ao atingir o limite, espera o *primeiro* worker terminar antes de submeter o próximo — RAM constante mesmo em 10 GB com N workers.

Limitação estrutural:

`pd.read_csv` lê *serialmente* no processo principal. O I/O é o gargalo — não o compute (ver Slide 16).



Por que "a média das médias" está errada

Estatísticas como média e desvio-padrão **não são lineares na concatenação**.

Chunk 1: `{10,10,10}` ($\mu=10$) · Chunk 2: `{20}` ($\mu=20$)

Correto (ponderado): $(3 \cdot 10 + 1 \cdot 20) / 4 = 12,5$

Média das médias: $(10+20)/2 = 15 \rightarrow$ erro de 20%

Estatísticas suficientes por chunk

Cada worker guarda apenas **2 escalares por coluna** — nenhum valor individual:

- $S_k = \sum x_i$ — soma dos valores do chunk
- $Q_k = \sum x_i^2$ — soma dos quadrados

Fórmulas de combinação global

$$\bar{x} = (\sum_k S_k) / (\sum_k n_k) \quad (1)$$

$$\sigma = \sqrt{(\sum_k Q_k / \sum_k n_k) - \bar{x}^2} \quad (2)$$

Desvio-padrão pela identidade de **König–Huygens**: $\text{Var}(X) = E[X^2] - (E[X])^2$ — *sem segunda passagem* pelos dados. Combinação em $O(K)$, $K = n^\circ$ de chunks.

Demais métricas (todas exatas):

min/max global (mín dos mins); *value_counts* = soma de `Counter`; valores únicos = união de `set`; nulos = soma; formato de data = 1º chunk não-nulo.

Garantia:

o sumário paralelo é *numericamente idêntico* ao do `DataSetSummary` original — compatibilidade total com o VisFlow-MM.

Como o Spark reexpressa o perfilamento

1. `SparkSession` em `local[N]` — N threads Java no mesmo processo, **sem GIL**
2. Leitura CSV paralela com inferência de schema
3. Catalyst [Zaharia et al. 2016] funde min/max/mean/stddev + `approx_count_distinct` em **um único job (scan único)**
4. **HyperLogLog** (rsd=0,05) para cardinalidade aproximada — ~10× mais rápido que `nunique` exato

```
num_exprs += [
    F.min(c), F.max(c), F.mean(c),
    F.stddev(c),
    F.approx_count_distinct(c, rsd=0.05)]
df.agg(*num_exprs).collect() # 1 job
```

Diferenças técnicas (Tabela 2 do artigo)

Aspecto	Multiprocessing	Apache Spark
Leitura CSV	1 thread serial (pandas)	Threads JVM II
Paralelismo	Processos Python	Threads Java (sem GIL)
Serialização	pickle (~90 MB/chunk)	memória off-heap
Agregações	passes múltiplos/chunk	1 job Catalyst
Cardinalidade	<code>nunique()</code> exato	HyperLogLog ±5%
Overhead inicial	≈0,1 s	10–15 s (JVM)*

* Warmup:

1 execução por configuração aquece JVM + cache do Catalyst e *não é contabilizada* no tempo medido.

Tungsten

(gestão de memória off-heap) e **AQE** (Adaptive Query Execution) reotimizam o plano físico em tempo de execução.

Hardware

Componente	Especificação
CPU	AMD Ryzen 7 8700G — 8 físicos / 16 lógicos (SMT)
RAM	64 GB DDR5
Disco	NVMe SSD (~3,5 GB/s seq.)
OS	Windows + Docker Desktop (WSL2)

Software

Ferramenta	Versão
Python · pandas	3.11 · 2.2.2
PySpark · Java	3.5.1 · OpenJDK 17
Docker Compose	v2 / Engine 26.x

Protocolo

Níveis de paralelismo:
 $N \in \{1, 2, 4, 8, 16\}$
Repetições: 3 por configuração (mínimo obrigatório)
Tamanhos: 1 GB · 5 GB · 10 GB
Chunk (MP): 500.000 linhas · **N=1** = loop serial puro (baseline)
Spark: 1 warmup/config (não contabilizado)

Métricas:
tempo wall-clock (leitura + agregações + coleta), vazão (MB/s) e speedup relativo (t_1/t_N).

Nota de ambiente:
Docker Desktop sobre WSL2 introduz overhead de virtualização no I/O. Em Linux nativo os tempos de leitura seriam menores. Tudo reproduzível via `docker compose` [Fonseca 2026a].

Tabela 3 — Tempo médio (s) e speedup por dataset · 3 repetições

Workers	1 GB — tempo	speedup	5 GB — tempo	speedup	10 GB — tempo	speedup
1 (serial)	23,48	1,00×	101,20	1,00×	192,27	1,00×
2	24,65	0,95×	96,14	1,05×	180,81	1,06×
4	25,17	0,93×	96,51	1,05×	182,25	1,06×
8	25,47	0,92×	96,81	1,04×	181,53	1,06×
16	25,40	0,92×	103,31	0,98×	181,65	1,06×

Speedup — 10 GB (1 worker = baseline)



RQ1 respondida — NÃO escala.

Em 1 GB, adicionar workers *piora* (speedup < 1): o overhead de criação de processos + pickle (~90 MB/chunk) supera qualquer ganho. Em 5/10 GB, platô em ~1,06×.

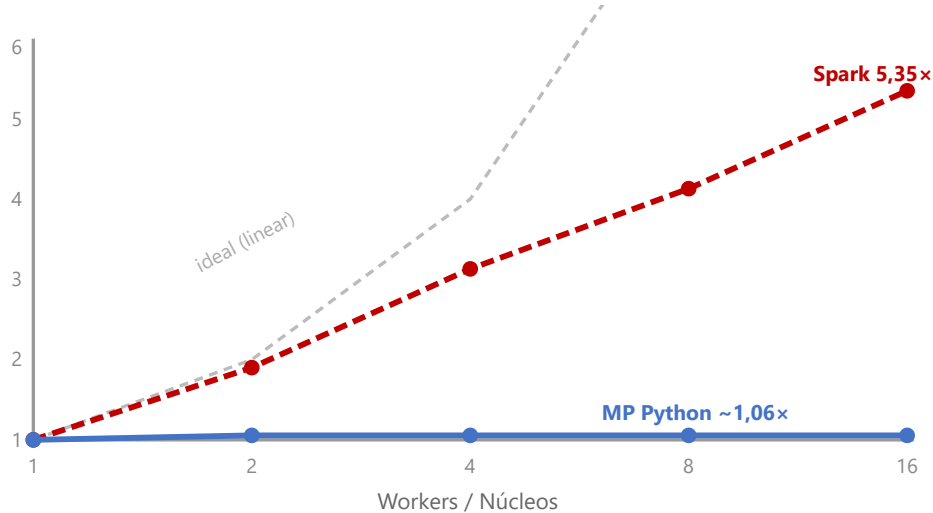
A vazão permanece travada em ~45–47 MB/s — o teto de leitura sequencial do SSD. O outlier em N=16 / 5 GB (113 s, $\sigma=8,76$) foi mantido por honestidade [Slide 17].

Apache Spark (Workload B) — Dataset 1 GB

Tabela 4 — Spark local[N], dataset 1 GB

Núcleos	Tempo (s)	Speedup	Efic. %	MB/s
1	124,55	1,00×	100,0	8,6
2	65,67	1,90×	94,9	16,3
4	39,76	3,13×	78,3	26,9
8	30,13	4,13×	51,7	35,5
16	23,28	5,35×	33,4	45,9

Curvas de speedup (Figura 3)



RQ2 respondida — ESCALA de verdade

- **Threads JVM sem GIL:** I/O e compute em paralelo real [Zaharia et al. 2016]
- **Catalyst:** todas as agregações em 1 scan físico
- **Escalada quase linear** de 1→4 cores (3,13×); desacelera depois (Amdahl)

Custo de entrada:

com 1 core, Spark (124,6 s) é **5,3× mais lento** que Python serial (23,5 s) — overhead de JVM, Tungsten e agendamento de tasks.

Ponto de cruzamento:

com 16 cores, Spark (23,3 s) *iguala* o Python serial (23,5 s) — mesmo throughput (~46 MB/s), por caminhos opostos. A eficiência cai de 100% → 33% (fração serial irreduzível).

$$S(n) = 1 / [F_s + (1 - F_s)/n] \quad \text{— speedup com } n \text{ unidades e fração serial } F_s \text{ [Amdahl 1967]}$$

(3)

Por que o Multiprocessing não escala

O processo principal lê o CSV a ~47 MB/s. O `profile_chunk()` termina em *décimos de segundo* — muito antes do disco entregar o próximo chunk. Os workers ficam **ociosos esperando dados**: workload *I/O-bound*. Soma-se o custo de pickle (~90 MB/chunk via IPC).

 $F_s \approx 90\%$ (leitura serial) $\rightarrow S(\infty) = 1/0,90 \approx 1,11\times$ Compatível com o observado ($\leq 1,06\times$).

Por que o Spark escala

Threads Java sem GIL executam I/O + CPU em paralelo real (o CPython bloqueia threads via GIL; processos evitam o GIL mas mantêm a leitura serial). O **Catalyst** funde todas as agregações em um único scan.

Ajustando aos dados ($S(16)=5,35$): **$F_s \approx 13,3\%$** $\rightarrow$ teto teórico $1/0,133 \approx 7,5\times$ O Spark *paraleliza o I/O* e derruba a fração serial de 90% $\rightarrow$ 13%.

Lição central:

paralelizar *computação* sem paralelizar *I/O* é insuficiente para workloads de leitura de arquivos locais. O ganho real exige atacar o gargalo dominante.

Recomendações, Validade e Conclusões

RQ3 — Guia de escolha (Tabela 5)

Cenário	Recomendação	Motivo
< 5 GB, máquina simples	Python chunks serial	Menor overhead
5–50 GB, 16+ cores	Spark <code>local[N]</code>	Escala com núcleos
> 50 GB ou cluster	Spark distribuído	I/O entre nós

Ameaças à validade

- **Interna:** máquina única com processos em 2º plano; outlier de N=16/5 GB mantido por honestidade
- **Externa:** específico ao SSD NVMe (~47 MB/s, 16 cores); em HDD o gargalo de I/O seria ainda mais dominante
- **Generalidade:** workload de leitura sem shuffles pesados — joins/shuffles favoreceriam ainda mais o Spark

Conclusões

1. **Problema resolvido:** ambas processam 10 GB sem `MemoryError`, RAM constante (~2 GB) via chunks + janela deslizando
2. **MP é I/O-bound:** speedup $\leq 1,06\times$ — a leitura serial limita tudo
3. **Spark escala:** $5,35\times$ com 16 cores; F_s de 90% $\rightarrow$ 13%
4. **Para o VisFlow-MM em produção:** Spark `local[N]` (16+ cores) ou cluster — com a ressalva do overhead de JVM em datasets pequenos

Trabalhos futuros

- Executar benchmark Spark para 5 GB e 10 GB
- Avaliar impacto do `chunksize` no multiprocessing
- Integrar o pipeline || como módulo opcional do VisFlow-MM, ativado por limiar de tamanho

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proc. AFIPS Spring Joint Computer Conf.*, p. 483–485.

Chen, N., Zhang, Y., Xu, J., Ren, K., Yang, Y. (2025). VisEval: A benchmark for data visualization in the era of large language models. *IEEE Trans. Vis. Comput. Graph.*, 31(1):1301–1311.

Dibia, V. (2023). LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using LLMs. *Proc. 61st ACL (System Demos)*, p. 113–126.

Fonseca, O. (2026a). Perfilamento paralelo com multiprocessing e Apache Spark: código-fonte dos experimentos. github.com/ootaviofonseca/pdm. PCC557/PDM/UFLA.

Fonseca, O. (2026b). VisFlow-MM: Agentic multimodal chart generation with vision-language models. github.com/ootaviofonseca/VISFLOW-MM. PPGCC/UFLA.

Maddigan, P., Susnjak, T. (2023). Chat2VIS: Generating data visualizations via natural language using ChatGPT, Codex and GPT-3 LLMs. *IEEE Access*, 11:45181–45193.

NYC Taxi & Limousine Commission (2024). TLC Trip Record Data. nyc.gov/site/tlc/about/tlc-trip-record-data.page. Acesso: jun. 2024.

Ouyang, G., Chen, J., Nie, Z., Gui, Y., Wan, Y., Zhang, H., Chen, D. (2025). NVAgent: Automated data visualization from natural language via collaborative agent workflow. *Proc. 63rd ACL*, p. 19534–19567.

Shuai, Z., Li, B., Yan, S., Luo, Y., Yang, W. (2026). DeepVIS: Bridging natural language and data visualization through step-wise reasoning. *IEEE Trans. Vis. Comput. Graph.*, 32(1):868–878.

Wu, Y., Wan, Y., Zhang, H., Sui, Y., Wei, W., Zhao, W., Xu, G., Jin, H. (2024). Automated data visualization from natural language via LLMs: An exploratory study. *Proc. ACM Manag. Data*, 2(3):115:1–115:28.

Zacks, J., Tversky, B. (1999). Bars and lines: A study of graphic communication. *Memory & Cognition*, 27(6):1073–1079.

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., et al. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65.

Zhu, Y., Du, S., Li, B., Luo, Y., Tang, N. (2024). Are large language models good statisticians? *Proc. 38th NeurIPS*.

Repositório público (código + Docker Compose + resultados + relatório HTML):

github.com/ootaviofonseca/pdm · Baseado no artigo *VisFlow-MM* [Fonseca 2026b]

MENSAGENS PARA LEVAR

Obrigado!

Perguntas?

1. O perfilamento (`DataSetSummary`) é o **teto de desempenho** de todo o pipeline NL2Vis — não é mera infraestrutura.

2. Chunks + agregação exata resolvem o `MemoryError` em 10 GB com RAM constante — sumário **numericamente idêntico** ao original.

3. Paralelizar *computação* sem paralelizar *I/O* não basta: o Multiprocessing satura ($\leq 1,06\times$), o Spark escala **5,35×** ao paralelizar o I/O.

github.com/otaviofonseca/pdm

VisFlow-MM [Fonseca 2026b]